

Kysely 日常語法大全

Kysely 是 TypeScript 的 type-safe SQL query builder。
本文件以日常 CRUD、後台 CMS、API 開發最常用的寫法為主。

目錄

- [1. SELECT](#)
 - [2. WHERE](#)
 - [3. ORDER BY](#)
 - [4. LIMIT / OFFSET](#)
 - [5. COUNT / SUM / AVG / MIN / MAX](#)
 - [6. GROUP BY / HAVING](#)
 - [7. JOIN](#)
 - [8. INSERT](#)
 - [9. UPDATE](#)
 - [10. DELETE](#)
 - [11. UPSERT](#)
 - [12. DISTINCT](#)
 - [13. CASE WHEN](#)
 - [14. COALESCE](#)
 - [15. Raw SQL](#)
 - [16. EXISTS](#)
 - [17. Subquery](#)
 - [18. CTE / WITH](#)
 - [19. Transaction](#)
 - [20. FOR UPDATE](#)
 - [21. Dynamic Query](#)
 - [22. Pagination](#)
 - [23. 常用 CRUD Cheat Sheet](#)
 - [24. 最常用 API](#)
-

1. SELECT

SELECT 全部欄位

```
const users = await db
  .selectFrom('users')
  .selectAll()
  .execute()
```

SQL :

```
SELECT *  
FROM users;
```

SELECT 指定欄位

```
const users = await db  
  .selectFrom('users')  
  .select([  
    'id',  
    'name',  
    'email',  
  ])  
  .execute()
```

SELECT 單一欄位

```
const users = await db  
  .selectFrom('users')  
  .select('id')  
  .execute()
```

Alias

```
const users = await db  
  .selectFrom('users')  
  .select([  
    'id',  
    'name as username',  
  ])  
  .execute()
```

SQL :

```
SELECT  
  id,  
  name AS username  
FROM users;
```

2. WHERE

等於

```
.where('status', '=', 'active')
```

不等於

```
.where('status', '!=', 'inactive')
```

大於

```
.where('age', '>', 18)
```

大於等於

```
.where('age', '>=', 18)
```

小於

```
.where('age', '<', 60)
```

小於等於

```
.where('age', '<=', 60)
```

AND

多個 `.where()` 預設就是 AND：

```
const users = await db
  .selectFrom('users')
  .selectAll()
  .where('status', '=', 'active')
  .where('age', '>=', 18)
  .execute()
```

SQL :

```
WHERE
  status = 'active'
AND age >= 18;
```

OR

```
const users = await db
  .selectFrom('users')
  .selectAll()
  .where((eb) =>
    eb.or([
      eb('status', '=', 'active'),
      eb('status', '=', 'inactive'),
    ])
  )
  .execute()
```

SQL :

```
WHERE
  status = 'active'
OR status = 'inactive';
```

AND + OR

```
.where((eb) =>
  eb.and([
    eb('age', '>=', 18),
    eb.or([
      eb('status', '=', 'active'),
      eb('status', '=', 'inactive'),
    ]),
  ])
)
```

SQL :

```
WHERE
  age >= 18
AND (
```

```
status = 'active'  
OR status = 'inactive'  
);
```

IN

```
.where('id', 'in', [1, 2, 3])
```

SQL :

```
WHERE id IN (1, 2, 3);
```

NOT IN

```
.where('id', 'not in', [1, 2, 3])
```

LIKE

```
.where('name', 'like', '%john%')
```

NOT LIKE

```
.where('name', 'not like', '%test%')
```

IS NULL

```
.where('age', 'is', null)
```

SQL :

```
WHERE age IS NULL;
```

IS NOT NULL

```
.where('age', 'is not', null)
```

BETWEEN

可以直接使用兩個條件：

```
.where('age', '>=', 18)  
.where('age', '<=', 60)
```

通常比複雜 expression 更容易閱讀。

欄位 vs 欄位

✗ Value 比較

```
.where('posts.user_id', '=', userId)
```

這是：

```
posts.user_id = ?
```

✓ Column 比較

```
.whereRef(  
    'posts.user_id',  
    '=',  
    'users.id'  
)
```

SQL：

```
posts.user_id = users.id
```

3. ORDER BY

ASC

```
.orderBy('name', 'asc')
```

DESC

```
.orderBy('created_at', 'desc')
```

多重排序

```
.orderBy('status', 'asc')  
.orderBy('created_at', 'desc')
```

SQL :

```
ORDER BY  
  status ASC,  
  created_at DESC;
```

4. LIMIT / OFFSET

LIMIT

```
.limit(20)
```

OFFSET

```
.offset(20)
```

完整例子

```
const users = await db  
  .selectFrom('users')  
  .selectAll()  
  .orderBy('id', 'desc')  
  .limit(20)
```

```
.offset(40)
.execute()
```

5. COUNT / SUM / AVG / MIN / MAX

COUNT

```
const result = await db
  .selectFrom('users')
  .select((eb) =>
    eb.fn.count('id').as('count')
  )
  .executeTakeFirst()
```

COUNT ALL

```
.select((eb) =>
  eb.fn.countAll().as('count')
)
```

COUNT + WHERE

```
const result = await db
  .selectFrom('users')
  .select((eb) =>
    eb.fn.count('id').as('count')
  )
  .where('status', '=', 'active')
  .executeTakeFirst()
```

COUNT DISTINCT

```
.select((eb) =>
  eb.fn
    .count('email')
    .distinct()
)
```



```
    .as('count')
  )
```

SUM

```
.select((eb) =>
  eb.fn.sum('amount').as('total')
)
```

AVG

```
.select((eb) =>
  eb.fn.avg('age').as('average_age')
)
```

MIN / MAX

```
.select((eb) => [
  eb.fn.min('age').as('min_age'),
  eb.fn.max('age').as('max_age'),
])
```

6. GROUP BY / HAVING

GROUP BY

```
const result = await db
  .selectFrom('users')
  .select([
    'status',
    (eb) =>
      eb.fn.count('id').as('count'),
  ])
  .groupBy('status')
  .execute()
```

SQL :

```
SELECT
  status,
  COUNT(id)
FROM users
GROUP BY status;
```

HAVING

```
const result = await db
  .selectFrom('users')
  .select([
    'status',
    (eb) =>
      eb.fn.count('id').as('count'),
  ])
  .groupBy('status')
  .having(
    (eb) => eb.fn.count('id'),
    '>',
    10
  )
  .execute()
```

7. JOIN

INNER JOIN

```
const result = await db
  .selectFrom('users')
  .innerJoin(
    'posts',
    'posts.user_id',
    'users.id'
  )
  .select([
    'users.id',
    'users.name',
    'posts.title',
  ])
  .execute()
```

SQL :

```
SELECT
  users.id,
  users.name,
  posts.title
FROM users
INNER JOIN posts
  ON posts.user_id = users.id;
```

LEFT JOIN

```
const result = await db
  .selectFrom('users')
  .leftJoin(
    'posts',
    'posts.user_id',
    'users.id'
  )
  .select([
    'users.id',
    'users.name',
    'posts.title',
  ])
  .execute()
```

RIGHT JOIN

```
.rightJoin(
  'posts',
  'posts.user_id',
  'users.id'
)
```

FULL JOIN

```
.fullJoin(
  'posts',
  'posts.user_id',
  'users.id'
)
```

JOIN 多個表

```
const result = await db
  .selectFrom('users')
  .leftJoin(
    'posts',
    'posts.user_id',
    'users.id'
  )
  .leftJoin(
    'categories',
    'categories.id',
    'posts.category_id'
  )
  .select([
    'users.id',
    'users.name',
    'posts.title',
    'categories.name as category_name',
  ])
  .execute()
```

JOIN 複合條件

```
.leftJoin('posts', (join) =>
  join
    .onRef(
      'posts.user_id',
      '=',
      'users.id'
    )
    .on(
      'posts.published',
      '=',
      true
    )
)
```

8. INSERT

INSERT 一筆

```
await db
  .insertInto('users')
  .values({
    name: 'John',
    email: 'john@example.com',
    status: 'active',
    age: 30,
    created_at: new Date(),
  })
  .execute()
```

INSERT 多筆

```
await db
  .insertInto('users')
  .values([
    {
      name: 'John',
      email: 'john@example.com',
      status: 'active',
      age: 30,
      created_at: new Date(),
    },
    {
      name: 'Mary',
      email: 'mary@example.com',
      status: 'active',
      age: 28,
      created_at: new Date(),
    },
  ])
  .execute()
```

INSERT + RETURNING

PostgreSQL 常用：

```
const user = await db
  .insertInto('users')
```

```
.values(data)
.returningAll()
.executeTakeFirstOrThrow()
```

RETURNING 指定欄位

```
const user = await db
  .insertInto('users')
  .values(data)
  .returning([
    'id',
    'name',
  ])
  .executeTakeFirstOrThrow()
```

9. UPDATE

UPDATE

```
await db
  .updateTable('users')
  .set({
    name: 'John Updated',
    age: 31,
  })
  .where('id', '=', userId)
  .execute()
```

UPDATE 單一欄位

```
await db
  .updateTable('users')
  .set('status', 'inactive')
  .where('id', '=', userId)
  .execute()
```

UPDATE + RETURNING

```
const user = await db
  .updateTable('users')
  .set({
    name: 'John',
  })
  .where('id', '=', userId)
  .returningAll()
  .executeTakeFirst()
```

UPDATE Expression

例如：

```
age = age + 1
```

Kysely：

```
await db
  .updateTable('users')
  .set((eb) => ({
    age: eb('age', '+', 1),
  })))
  .where('id', '=', userId)
  .execute()
```

10. DELETE

DELETE

```
await db
  .deleteFrom('users')
  .where('id', '=', userId)
  .execute()
```

DELETE 多筆

```
await db
  .deleteFrom('users')
  .where('status', '=', 'inactive')
  .execute()
```

DELETE + RETURNING

```
const deleted = await db
  .deleteFrom('users')
  .where('id', '=', userId)
  .returningAll()
  .executeTakeFirst()
```

11. UPSERT

PostgreSQL / SQLite

```
await db
  .insertInto('users')
  .values({
    email: 'john@example.com',
    name: 'John',
    status: 'active',
    age: 30,
    created_at: new Date(),
  })
  .onConflict((oc) =>
    oc
      .column('email')
      .doUpdateSet({
        name: 'John',
        status: 'active',
      })
  )
  .execute()
```

ON CONFLICT DO NOTHING

```
await db
  .insertInto('users')
  .values(data)
  .onConflict((oc) =>
    oc
      .column('email')
      .doNothing()
  )
  .execute()
```

12. DISTINCT

```
const users = await db
  .selectFrom('users')
  .select('email')
  .distinct()
  .execute()
```

DISTINCT 多欄

```
const users = await db
  .selectFrom('users')
  .select([
    'name',
    'email',
  ])
  .distinct()
  .execute()
```

13. CASE WHEN

```
const result = await db
  .selectFrom('users')
  .select((eb) =>
    eb
      .case()
      .when('age', '>=', 18)
      .then('adult')
```

```
        .else('child')
        .end()
        .as('age_group')
    )
    .execute()
```

SQL :

```
CASE
  WHEN age >= 18 THEN 'adult'
  ELSE 'child'
END AS age_group
```

Status 顯示文字

```
.select((eb) =>
  eb
    .case()
    .when('status', '=', 'active')
    .then('正常')
    .else('停用')
    .end()
    .as('status_text')
)
```

14. COALESCE

NULL 顯示預設值：

```
.select((eb) =>
  eb.fn
    .coalesce('age', 0)
    .as('age')
)
```

SQL :

```
COALESCE(age, 0)
```

15. Raw SQL

需要使用資料庫特有功能時，可以使用 `sql`。

```
import { sql } from 'kysely'
```

Raw SQL

```
const result = await db
  .selectFrom('users')
  .select([
    'id',
    sql<string>`LOWER(name)` .as('lower_name'),
  ])
  .execute()
```

Raw SQL + Parameter

✓ 正確

```
sql`price * ${rate}`
```

Kysely 會處理 parameter binding。

✗ 不要

```
sql.raw(`price * ${rate}`)
```

尤其不要將 user input 直接拼進 SQL。

SQL Function

```
sql`NOW()`
```

SQL Reference

```
sql.ref('users.name')
```

SQL Value

```
sql.val(value)
```

16. EXISTS

找出「至少有一篇文章」的 users：

```
const users = await db
  .selectFrom('users')
  .selectAll()
  .where((eb) =>
    eb.exists(
      eb
        .selectFrom('posts')
        .select('posts.id')
        .whereRef(
          'posts.user_id',
          '=',
          'users.id'
        )
    )
  )
  .execute()
```

SQL：

```
WHERE EXISTS (
  SELECT posts.id
  FROM posts
  WHERE posts.user_id = users.id
)
```

NOT EXISTS

```
.where((eb) =>
  eb.not(
    eb.exists(
      eb
        .selectFrom('posts')
        .select('id')
        .whereRef(
          'posts.user_id',
          '=',
          'users.id'
        )
    )
  )
)
```

17. Subquery

例如：

```
const users = await db
  .selectFrom('users')
  .selectAll()
  .where(
    'id',
    'in',
    db
      .selectFrom('posts')
      .select('user_id')
  )
  .execute()
```

SQL：

```
WHERE id IN (
  SELECT user_id
  FROM posts
)
```

18. CTE / WITH

```
const result = await db
  .with('active_users', (db) =>
    db
      .selectFrom('users')
      .selectAll()
      .where(
        'status',
        '=',
        'active'
      )
  )
  .selectFrom('active_users')
  .selectAll()
  .execute()
```

SQL :

```
WITH active_users AS (
  SELECT *
  FROM users
  WHERE status = 'active'
)
SELECT *
FROM active_users;
```

多個 CTE

```
const result = await db
  .with('active_users', (db) =>
    db
      .selectFrom('users')
      .selectAll()
      .where(
        'status',
        '=',
        'active'
      )
  )
  .with('published_posts', (db) =>
    db
      .selectFrom('posts')
      .selectAll()
      .where(
```

```

        'published',
        '=',
        true
    )
)
.selectFrom('active_users')
.innerJoin(
    'published_posts',
    'published_posts.user_id',
    'active_users.id'
)
.selectAll()
.execute()

```

19. Transaction

非常適合：

- 建立訂單
- 扣庫存
- 建立付款資料
- 建立會員 + Profile
- 多張表同步更新

```

await db.transaction().execute(async (trx) => {

    const user = await trx
        .insertInto('users')
        .values(data)
        .returningAll()
        .executeTakeFirstOrThrow()

    await trx
        .insertInto('posts')
        .values({
            user_id: user.id,
            title: 'Hello',
            content: 'World',
            published: true,
            created_at: new Date(),
        })
        .execute()

})

```

成功：

```
COMMIT
```

任何一步發生錯誤：

```
ROLLBACK
```

Transaction Isolation

```
await db
  .transaction()
  .setIsolationLevel('serializable')
  .execute(async (trx) => {
    // ...
  })
```

20. FOR UPDATE

適合：

- 庫存
- 訂單
- 付款
- 票券
- 排隊
- 避免 concurrent update

```
const order = await trx
  .selectFrom('orders')
  .selectAll()
  .where('id', '=', orderId)
  .forUpdate()
  .executeTakeFirst()
```

概念：

```
SELECT *
FROM orders
WHERE id = ?
FOR UPDATE;
```


21. Dynamic Query

後台 Search / Filter 最常用。

```
let query = db
  .selectFrom('users')
  .selectAll()

if (keyword) {
  query = query.where(
    'name',
    'like',
    `%${keyword}%`
  )
}

if (status) {
  query = query.where(
    'status',
    '=',
    status
  )
}

if (minAge) {
  query = query.where(
    'age',
    '>=',
    minAge
  )
}

const users = await query.execute()
```

Dynamic Filter 實戰

```
let query = db
  .selectFrom('users')
  .select([
    'id',
    'name',
    'email',
    'status',
  ])

if (keyword) {
```

```

query = query.where((eb) =>
  eb.or([
    eb(
      'name',
      'like',
      `%${keyword}%`
    ),
    eb(
      'email',
      'like',
      `%${keyword}%`
    ),
  ])
)
}

if (status) {
  query = query.where(
    'status',
    '=',
    status
  )
}

query = query
  .orderBy('created_at', 'desc')
  .limit(20)

const result = await query.execute()

```

22. Pagination

基本 Pagination

```

const page = 3
const pageSize = 20

const users = await db
  .selectFrom('users')
  .selectAll()
  .orderBy('id', 'desc')
  .limit(pageSize)
  .offset(
    (page - 1) * pageSize
  )
  .execute()

```

Pagination + Count

```
const [rows, countResult] =
  await Promise.all([
    db
      .selectFrom('users')
      .selectAll()
      .orderBy('id', 'desc')
      .limit(pageSize)
      .offset(
        (page - 1) * pageSize
      )
      .execute(),

    db
      .selectFrom('users')
      .select((eb) =>
        eb.fn
          .countAll()
          .as('total')
      )
      .executeTakeFirstOrThrow(),
  ])
```

23. 常用 CRUD Cheat Sheet

SELECT

```
db
  .selectFrom('users')
  .selectAll()
  .execute()
```

SELECT ONE

```
db
  .selectFrom('users')
  .selectAll()
  .where('id', '=', id)
  .executeTakeFirst()
```

SELECT ONE OR THROW

```
db
  .selectFrom('users')
  .selectAll()
  .where('id', '=', id)
  .executeTakeFirstOrThrow()
```

INSERT

```
db
  .insertInto('users')
  .values(data)
  .execute()
```

INSERT + RETURN

```
db
  .insertInto('users')
  .values(data)
  .returningAll()
  .executeTakeFirstOrThrow()
```

UPDATE

```
db
  .updateTable('users')
  .set(data)
  .where('id', '=', id)
  .execute()
```

UPDATE + RETURN

```
db
  .updateTable('users')
  .set(data)
  .where('id', '=', id)
```

```
.returningAll()
.executeTakeFirst()
```

DELETE

```
db
  .deleteFrom('users')
  .where('id', '=', id)
  .execute()
```

DELETE + RETURN

```
db
  .deleteFrom('users')
  .where('id', '=', id)
  .returningAll()
  .executeTakeFirst()
```

24. 最常用 API

以下這些建議直接記住：

```
selectFrom()
select()
selectAll()

where()
whereRef()

innerJoin()
leftJoin()
rightJoin()
fullJoin()

orderBy()
groupBy()
having()

limit()
offset()
```

```
insertInto()  
values()  
returning()  
returningAll()  
onConflict()  
  
updateTable()  
set()  
  
deleteFrom()  
  
execute()  
executeTakeFirst()  
executeTakeFirstOrThrow()  
  
transaction()  
  
forUpdate()  
  
with()  
  
exists()  
  
sql`...`
```

Kysely 日常 Query 心法

```
SELECT  
  ↓  
selectFrom()  
  ↓  
select()  
  ↓  
where()  
  ↓  
join()  
  ↓  
groupBy()  
  ↓  
having()  
  ↓  
orderBy()  
  ↓  
limit()  
  ↓  
offset()
```

↓
execute()

CRUD :

CREATE → insertInto() → values()

READ → selectFrom() → select()

UPDATE → updateTable() → set()

DELETE → deleteFrom()

複雜 SQL :

JOIN
↓
Subquery
↓
EXISTS
↓
CTE / WITH
↓
CASE
↓
sql`...`

資料一致性 :

transaction()
↓
forUpdate()
↓
COMMIT / ROLLBACK

90% 日常開發版本

如果只想記最重要的 :

```
// SELECT
await db
  .selectFrom('users')
  .selectAll()
```

```

    .where('id', '=', id)
    .executeTakeFirst()

// INSERT
await db
    .insertInto('users')
    .values(data)
    .returningAll()
    .executeTakeFirstOrThrow()

// UPDATE
await db
    .updateTable('users')
    .set(data)
    .where('id', '=', id)
    .returningAll()
    .executeTakeFirst()

// DELETE
await db
    .deleteFrom('users')
    .where('id', '=', id)
    .execute()

// JOIN
await db
    .selectFrom('users')
    .leftJoin(
        'posts',
        'posts.user_id',
        'users.id'
    )
    .selectAll()
    .execute()

// TRANSACTION
await db.transaction().execute(async (trx) => {
    // multiple queries
})

// RAW SQL
sql`...`

```

一句話記憶

Kysely = SQL 思維 + TypeScript Type Safety

如果你識 SQL，Kysely 最重要不是背語法，而是熟習：


```
selectFrom  
where  
whereRef  
join  
orderBy  
limit  
insertInto  
updateTable  
deleteFrom  
transaction  
sql
```

這幾組已經可以覆蓋絕大部分日常後台 / API CRUD。